UNILANG PLATFORM ENGINEERING & ARCHITECTURE

UniLang Master Build Prompts

Actionable Prompts for Autonomous End-to-End Module Development

DOCUMENT EXECUTIVE PURPOSE

This catalog contains six production-grade, comprehensive Master Build Prompts designed to direct an AI coding assistant to build all missing UniLang modules end-to-end. Each prompt includes full architectural context, strict acceptance criteria, file paths, and testing verification commands.

Specification Attribute	Document Metadata
Document ID	UNILANG-PROMPT-2026-V1
Document Version	1.0 Final
Classification	Engineering Prompt Catalog & Execution Guide
Target Systems	UniLang Web App (React 18 + Vite), Future Backend & AI Orchestrator
Publication Date	November 2026
Repository Target	c:\Users\yadav\OneDrive\Desktop\UNILANG
Implementation Status	Ready for Immediate Execution

CRITICAL EXECUTIVE SUMMARY HIGHLIGHTS

- Master Prompt 1: Persistent PostgreSQL Database & Backend Server Setup
- Master Prompt 2: Complete CEFR Learning Engine & Multi-Modal Quiz Studio
- Master Prompt 3: AI Scenario Conversation Partner & Voice Engine
- Master Prompt 4: Spaced Repetition (SRS) Vocabulary Builder & Flashcards
- Master Prompt 5: Interactive Grammar Coach & Verb Conjugation Tool

How to Use These Master Prompts

Each prompt in this document is engineered to be an all-in-one execution directive. You can copy the exact text within the callout boxes and paste it into the Antigravity chat interface. The assistant will autonomously build the database tables, API routes, React components, state hooks, and testing suites.

Master Prompt 1: Persistent PostgreSQL Database & Backend Server

Target Subsystems: /server, PostgreSQL, Prisma ORM, JWT Authentication, REST APIs.

PROMPT 1 (COPY & PASTE)

Act as the Lead Full-Stack Architect for the UniLang platform.
TASK: Build the complete backend persistence layer and API server to replace all mock client-side state.
REQUIREMENTS:
1. Setup a Node.js/Fastify (or Express) server in /server with TypeScript, cors, dotenv, and zod.
2. Initialize Prisma ORM in /server/prisma/schema.prisma with PostgreSQL models: User, UserPreference, Language, LearningPath, Lesson, Exercise, UserProgress, VocabularyItem, SavedVocabulary, TranslationHistory, ConversationSession, PhrasebookEntry. Include all relations, foreign keys, and compound indexes documented in Section 2 of UNILANG-ARCH-2026-V1.
3. Build authentication routes in /server/src/routes/auth.ts (register, login, refresh, me) using bcrypt password hashing and JWT cookies. Ensure the 1-click demo user (demo@unilang.app) is automatically seeded.
4. Build REST API endpoints for /api/lessons, /api/user/progress, and /api/vocabulary.
5. Create a client-side API client in /src/api/client.js that seamlessly swaps localStorage with authenticated server API calls.
VERIFICATION: Run migrations, seed test data, verify all endpoints with automated curl/supertest checks, and ensure the frontend loads live database records.

Master Prompt 2: Complete CEFR Learning Engine & Quiz Studio

Target Subsystems: /src/components/learn/, Multi-Modal Exercises, Placement Test.

PROMPT 2 (COPY & PASTE)

Act as the Senior Frontend & Curriculum Engineer for UniLang.
TASK: Transform the partial LearnSection.jsx into an exhaustive, multi-tier CEFR curriculum engine.
REQUIREMENTS:
1. Create a structured curriculum database in /src/data/curriculum/ with full A1 and A2 modules for Spanish, French, German, Italian, and Japanese (Greetings, Dining, Travel, Hotel, Directions, Shopping, Daily Routine).
2. Build /src/components/learn/PlacementTestModal.jsx: An adaptive 15-question placement assessment evaluating vocabulary, grammar, and listening comprehension to assign user starting level.
3. Overhaul QuizModal.jsx to support 5 distinct exercise interaction types:
   a) Multiple Choice with audio prompts and explanation callouts.
   b) Sentence Reorder: Interactive draggable/clickable word chips.
   c) Fill-in-the-Blank with contextual grammar hints.
   d) Listen & Transcribe with speed-adjustable audio playback.
   e) Pronunciation Speaking match with mic recording.
4. Implement a comprehensive lesson completion screen showing XP gained, accuracy percentage, time spent, and new vocabulary unlocked.
VERIFICATION: Test all 5 exercise types; ensure progress and XP increment the user dashboard; verify responsive mobile touch behavior.

Master Prompt 3: AI Scenario Conversation Partner & Voice Engine

Target Subsystems: /src/components/speak/, Web Speech API, AI Provider Adapter.

PROMPT 3 (COPY & PASTE)

Act as the Principal AI Engineer for UniLang.
TASK: Build the end-to-end AI Conversation Partner module in /src/components/speak/ConversationPartner.jsx.
REQUIREMENTS:
1. Create server-side endpoint POST /api/ai/chat that interfaces with the AI provider (Gemini or OpenAI) using the system prompt defined in Section 3.1 of UNILANG-ARCH-2026-V1.
2. The AI must return a structured payload containing: reply text, English translation, Romanized phonetic guide, and non-intrusive grammar corrections.
3. Implement 6 selectable roleplay scenarios: Italian Café, Paris Metro, Tokyo Hotel Check-in, Berlin Job Interview, Madrid Tapas Bar, Mexico City Taxi.
4. Build a conversational UI with message bubbles, audio play buttons for each AI turn (using Web Speech API or Cloud TTS), and an expandable "Grammar Coach Feedback" sidebar.
5. Implement speech-to-text input via the browser Web Speech API so learners can speak their responses directly.
6. Provide an end-of-session summary modal rating fluency, grammar accuracy, and vocabulary variety.
VERIFICATION: Verify conversation flow, test scenario switching, confirm grammar corrections render without breaking chat flow.

Master Prompt 4: Spaced Repetition (SRS) Vocabulary Builder

Target Subsystems: /src/components/vocabulary/, SuperMemo SM-2 Engine, Flashcards.

PROMPT 4 (COPY & PASTE)

Act as the Senior Full-Stack Engineer for UniLang.
TASK: Build the complete Vocabulary Builder & Spaced Repetition (SRS) engine.
REQUIREMENTS:
1. Implement the SuperMemo SM-2 algorithm in /src/lib/srs.js:
   - Calculate interval (I), repetition count (n), and ease factor (EF) based on user recall grade (0 = complete blackout to 5 = perfect recall).
   - Compute next review due date and retention strength percentage.
2. Build /src/components/vocabulary/VocabularySection.jsx featuring:
   a) Daily Review Queue: Displays cards due for review today.
   b) Flashcard Player: 3D flip card animation showing term, phonetics, audio, example sentence, and recall rating buttons (Again, Hard, Good, Easy).
   c) Master Word Deck: Searchable, filterable table of all saved words with mastery indicators.
   d) "Add to Vocabulary" action: Wire this up across Culture Explorer, Translator, and Lessons so any highlighted word can be saved with one tap.
3. Provide offline caching via IndexedDB/localStorage so flashcard reviews can be completed without internet connectivity.
VERIFICATION: Test SM-2 mathematical intervals with unit tests; verify card flipping and database sync.

Master Prompt 5: Interactive Grammar Coach & Conjugation Tool

Target Subsystems: /src/components/grammar/, Verb Conjugation Tables, Syntax Parser.

PROMPT 5 (COPY & PASTE)

Act as the Lead Linguistic Tooling Engineer for UniLang.
TASK: Build the Interactive Grammar Coach & Conjugation Engine in /src/components/grammar/GrammarSection.jsx.
REQUIREMENTS:
1. Build dynamic verb conjugation tables for regular and irregular verbs across Present, Preterite, Imperfect, Future, Conditional, and Subjunctive tenses for Spanish, French, Italian, German, and Portuguese.
2. Create an interactive "Sentence Analyzer" input where learners type or paste a sentence in the target language:
- Send input to POST /api/ai/analyze-grammar (implementing Prompt 3.2 from UNILANG-ARCH-2026-V1).
- Display colored token badges for each part of speech (Subject, Verb, Object, Preposition).
- Flag grammatical errors with clear linguistic explanations and one-click corrections.
3. Build a curated Grammar Rule Library organized by CEFR levels (e.g. "Ser vs Estar", "Subjunctive Mood Triggers", "Direct vs Indirect Pronoun Order").
VERIFICATION: Verify conjugation accuracy against standard linguistic models; ensure sentence parser catches tense and gender agreement errors.

Master Prompt 6: Offline Travel Phrasebook & Reading Studio

Target Subsystems: /src/components/phrasebook/, /src/components/reading/, Audio Sync.

PROMPT 6 (COPY & PASTE)

Act as the Senior Frontend Engineer for UniLang.
TASK: Build the Offline Travel Phrasebook and the Reading & Listening Studio.
REQUIREMENTS:
1. Build /src/components/phrasebook/PhrasebookSection.jsx:
- Implement the 8 core categories (Emergency, Medical, Food, Directions, Lodging, Transit, Shopping, Social) with 50+ essential phrases per language.
- Include rapid-access "Emergency Mode" pinned at the top with red accent styling.
- Every phrase must include: English, target script, Romanized phonetics, audio playback button, and "Save to Notebook" icon.
- Instant search filter indexing English, target script, and phonetics.
2. Build /src/components/reading/ReadingStudio.jsx:
- Dual-language parallel reader displaying side-by-side or stacked synchronized sentences.
- Interactive audio player with sentence-level karaoke highlighting as audio advances.
- Interactive word tokens: Clicking any word opens a quick definition popover.
- 3-question reading comprehension quiz at the end of each passage.
VERIFICATION: Verify audio-text karaoke synchronization; ensure offline phrasebook accessibility; test mobile responsiveness.